Shofe: Cursor AI Build Prompt, Night Owl Mode, APIs and Pesepay

Companion to the Shofe product documentation. Copy the master prompt (Section 1) into Cursor, and save the rules (Section 2) as project rules.

Assumptions: "Night Owl" is read as a set of night-time and anywhere-assistance features for drivers (and passengers). "Pese-pay" is read as **Pesepay**, the Zimbabwe-based payment gateway. Tell me if either is wrong.

1. Master Prompt for Cursor (paste into Agent/Composer)

Use this once at the start. Then work phase by phase (see Section 6) instead of asking Cursor to build everything at once.

```
You are a senior full-stack and mobile engineer. Help me build "Shofe", a
global, negotiation-based ride-hailing platform (inDrive-style). Passengers
propose a fare, nearby drivers accept or counter-offer, the passenger picks
an offer, and the agreed price is locked for the trip.

## GOAL
Production-quality MVP that works first in Zimbabwe and is architected to
scale globally (multi-currency, multi-language, multi-country, multiple
payment providers).

## USERS
- Passenger app, Driver app, Admin web dashboard.

## TECH STACK (use exactly this unless you ask me first)
- Mobile: Flutter (Dart) for Passenger and Driver apps, shared packages for
models, networking, theming.
- Backend: Node.js + TypeScript with NestJS, REST + WebSockets (Socket.IO).
- Database: PostgreSQL with PostGIS. Redis for driver locations, caching,
queues (BullMQ).
- Admin: React + TypeScript + Vite + Tailwind.
- Infra: Docker, docker-compose for local dev, environment-based config, CI
via GitHub Actions.
- Monorepo layout: /apps/passenger_app, /apps/driver_app, /apps/admin_web,
/services/api, /packages/shared, /infra, /docs.
```

```
## CORE DOMAIN RULES
1. Ride request: pickup, destination, offered_fare, currency,
vehicle_category.
2. Dispatch: find online drivers within an expanding radius (3km, 5km, 8km)
using PostGIS/H3; rank by distance, rating, acceptance rate; broadcast the
request over WebSockets and push notifications.
3. Offers: a driver can ACCEPT the offered fare or COUNTER with a price
inside min/max guardrails (min fare, max 2x suggested fare). Passenger sees
all offers (price, ETA, rating, vehicle) and selects one.
4. Once selected, the fare is LOCKED. Changing it requires explicit consent
from both sides.
5. Trip state machine: REQUESTED, OFFERED, MATCHED, DRIVER_EN_ROUTE, ARRIVED,
IN_PROGRESS, COMPLETED, CANCELLED. Enforce legal transitions server-side and
log every transition.
6. Pickup verification with a 4-digit PIN.
7. Ratings both ways. Strike and suspension rules for drivers; temporary
blocks for abusive passengers.
8. Driver onboarding: phone OTP, document upload (ID, licence, vehicle
registration, insurance, selfie, vehicle photos), admin approval queue,
document-expiry tracking with automatic blocking.
9. Commission: a configurable percentage per country and per driver tier,
recorded on every completed trip.
10. Driver commission wallet: cash trips deduct commission from a prepaid
driver wallet; if the balance is below a configurable threshold the driver
cannot receive new requests until topped up.

## NIGHT OWL MODE (night-time safety and anywhere assistance)
Implement as a dedicated module (api: /services/api/src/night-owl, driver
app: /apps/driver_app/lib/features/night_owl). See the feature list in the
NIGHT OWL section of my docs and implement:
- Automatic activation by local sunset/sunrise at the driver's GPS location,
with manual toggle.
- Safety check-ins, fatigue and long-shift reminders, SOS with live location
sharing, Guardian contacts, route-deviation and long-stop alerts, quick-help
finder (nearest fuel, hospital, police, mechanic, safe 24h spots), country-
aware emergency numbers, offline fallback, and a high-contrast dark UI.

## PAYMENTS (important)
Use a payment-provider abstraction: interface PaymentProvider { initiate(),
confirm(), checkStatus(), refund(), handleWebhookOrPoll() }.
- Implement PesepayProvider first (Zimbabwe: mobile money, cards). Add a
StripeProvider stub for global cards and a provider registry chosen by
country and currency.
```

```
- Pesepay uses an integration key and an encryption key. NEVER put them in the mobile apps. All payment calls happen on the backend only; keys come from environment variables/secret manager.
- Support both Pesepay flows: (a) redirect/web transaction (initiate, send user to the redirect URL, handle the result/return URL) and (b) seamless payment (get active currencies and payment methods by currency, collect required fields, submit). Store the reference number and poll URL for each transaction and poll status until final (use a background job with backoff).
- Make every payment operation idempotent, keep an immutable ledger table (double-entry style) for wallet and commission movements, and never trust the client for amounts.
- Configure result_url and return_url per environment.

## GLOBAL READINESS
- Multi-currency (store money as integer minor units + ISO currency code, never floats).
- i18n/l10n from day one (English first, Shona, Ndebele, French, Portuguese, Swahili, Arabic with RTL ready).
- Per-country config table: currency, commission %, min fare, vehicle categories, payment providers, emergency numbers, required driver documents, tax settings.
- Time zones stored in UTC, displayed locally. Distance units configurable (km/mi).

## NON-FUNCTIONAL REQUIREMENTS
- Low-data, low-end-Android friendly: small payloads, image compression, retry/queue on weak networks, SMS fallback for critical events.
- Security: JWT with refresh tokens, OTP rate limiting, input validation (class-validator), RBAC, encrypted document storage, signed URLs, audit logs, no secrets in code.
- Observability: structured logging, Sentry, health endpoints.
- Testing: unit tests for pricing, state machine, wallet ledger and dispatch; integration tests for the ride flow; payment provider tests with mocked responses.

## HOW TO WORK WITH ME
1. First, propose the folder structure, database schema (SQL migrations) and API contract (OpenAPI). Wait for my approval.
2. Then implement ONE phase at a time. After each phase: list files created, how to run it, and what to test.
3. Ask me before adding any library not listed. Prefer small, typed, well-tested modules.
4. Do not invent API endpoints, SDK methods or response fields for third-
```

```
party services; if unsure, tell me what to verify in the provider's docs.
5. Keep all code and comments in English; keep user-facing text in
localization files.

Start now with Phase 0: repo scaffolding, docker-compose (Postgres+PostGIS,
Redis), the DB schema, and the OpenAPI contract for auth, driver onboarding,
ride requests, offers and trips.
```

2. Project Rules (save as `.cursor/rules/shofe.mdc`)

```
---
description: Shofe project conventions
alwaysApply: true
---
- Language: TypeScript (strict) on backend/admin, Dart (null-safe) in
Flutter.
- Money: integer minor units + ISO 4217 currency code. No floating point.
- Time: UTC in DB and API (ISO 8601); convert for display only.
- Every endpoint: DTO validation, auth guard, role check, tests.
- Trip and payment status changes go through the state machine / ledger
services only.
- Secrets only via environment variables. Never log secrets, tokens, OTPs or
full phone numbers.
- Third-party SDK calls live behind interfaces in
/services/api/src/integrations/*.
- No direct payment calls from mobile apps.
- Add or update tests with every change. Small commits, clear messages.
- When unsure about an external API, say so and point to the docs instead of
guessing.
```

3. Night Owl Mode: Feature Specification

Purpose: assist a driver safely and practically at night, and wherever in the world they happen to be.

#	Feature	What it does
1	Auto Night Mode	Switches on at local sunset (from GPS) and off at sunrise; dark, high-contrast UI with large buttons

#	Feature	What it does
2	Safety Check-ins	Periodic "Are you OK?" prompt on night trips; no response triggers escalation to Guardian and support
3	SOS Button	One tap sends live location, trip details and vehicle plate to Guardian contacts, Shofe support and (optionally) local emergency services
4	Guardian Contacts	Trusted people the driver chooses; they receive a live-tracking link during night shifts
5	Fatigue and Shift Alerts	Reminds the driver to rest after long continuous driving; optional enforced break suggestion
6	Route Deviation and Long-Stop Alerts	Flags unexpected detours or long stationary periods mid-trip
7	Quick-Help Finder	Nearest 24-hour fuel station, hospital/clinic, police station, mechanic/tow, safe well-lit parking, wherever the driver is
8	Country-Aware Emergency Numbers	Shows the right emergency number for the driver's current country and one-tap dial
9	Breakdown Assistance	"I'm stuck" flow: share location, request tow/mechanic from partners, notify the passenger and reassign the trip
10	Safe-Zone Guidance	Highlights well-lit, busier pickup spots; warns about areas flagged by community reports
11	Offline Fallback	Caches last known map area, emergency numbers and Guardian contacts; SOS falls back to SMS when data is unavailable
12	Night Passenger Verification	Extra PIN/photo match at pickup after dark; passenger trust score shown to the driver
13	Night Earnings Boost (optional)	Driver can mark preference for night premium; the final price is still negotiated, never auto-surged

#	Feature	What it does
14	Low-Battery Protection	Warns at low battery, reduces GPS frequency smartly, prompts to charge before accepting trips
15	24/7 Support Chat and Call	Priority support line during night hours
16	Weather and Road Alerts	Fog, heavy rain or road-closure warnings along the route

Backend module (`night-owl`): night-mode scheduler, check-in service, SOS dispatcher (push, SMS, email), Guardian link generator (expiring tokens), place-finder proxy (cached), incident log, admin escalation queue.

Privacy rule: Guardians see location only while a night trip or active SOS is running, and the driver can revoke access at any time.

4. Recommended APIs and Services (global-ready)

4.1 Maps, Location and Routing

Need	Recommended	Alternative	Notes
Maps display	Google Maps SDK (Flutter)	Mapbox	Google has the best global coverage and Places data
Directions, ETA, distance	Google Routes API	Mapbox Directions, OSRM or Valhalla (self-hosted)	Self-host later to cut cost at scale
Geocoding and address search	Google Places and Geocoding	Mapbox Search, Nominatim	Cache aggressively
Nearby services (fuel, hospital, police)	Google Places Nearby Search	Foursquare Places, OpenStreetMap Overpass	Powers Night Owl Quick-Help Finder
Geospatial queries	PostGIS	H3 (Uber's hex grid)	H3 is excellent for dispatch zones

Need	Recommended	Alternative	Notes
Sunrise/sunset	Computed locally from lat/long (suncalc library)	Sunrise-Sunset API	No external call needed

4.2 Authentication, Messaging and Notifications

Need	Recommended	Alternative
Phone OTP	Twilio Verify	Firebase Auth Phone, Africa's Talking (strong in Africa)
SMS (alerts, SOS fallback)	Twilio, Africa's Talking	Vonage, Infobip
Push notifications	Firebase Cloud Messaging	OneSignal
Masked calls and chat	Twilio Proxy / Voice	Vonage Masked Numbers
Email	SendGrid or Amazon SES	Resend

4.3 Real-Time and Backend Infrastructure

Need	Recommended	Alternative
Real-time channel	Socket.IO on NestJS	Ably, Pusher, Firebase Realtime DB
Queues and jobs	Redis + BullMQ	AWS SQS
File storage (documents, photos)	S3-compatible storage (AWS S3, Cloudflare R2)	Google Cloud Storage
Hosting	AWS, DigitalOcean or Google Cloud	Render, Fly.io for early stage
Monitoring	Sentry + Grafana/Prometheus	Datadog

4.4 Identity Verification and Safety

Need	Recommended	Alternative
Driver KYC (ID, selfie, liveness)	Smile Identity (Africa focus)	Onfido, Veriff, Persona
Document OCR	Google Cloud Vision	AWS Textract
Fraud and risk	Device checks + rules engine	Sift, Stripe Radar (cards)
Weather alerts	OpenWeatherMap	Tomorrow.io
Currency rates	Open Exchange Rates	exchangerate.host, Fixer

4.5 Payments (summary)

Region	Provider
Zimbabwe (launch)	**Pesepay** (mobile money and cards)
Global cards and wallets	Stripe (or Adyen later)
Pan-African expansion	Flutterwave, Paystack
Cash	Driver commission wallet (no gateway needed)

Advice: wrap every provider behind one `PaymentProvider` interface (included in the master prompt) so you can add countries without rewriting your payment code.

5. Pesepay Integration Notes

Based on Pesepay's public SDK pages, here is what to expect (always confirm against your Pesepay merchant documentation and dashboard, since details can change):

- **Credentials:** an **integration key** and an **encryption key**, issued to you as a merchant.
- **Two payment flows:** a **redirect (web) transaction**, where you initiate a transaction and send the customer to a Pesepay page; and a **seamless payment**, where your app collects the payment-method details and submits them directly.
- **Currencies and methods:** SDKs expose calls to list active currencies and the payment methods available for a currency, so build your payment screen dynamically from these.

- **Status tracking:** each transaction returns a **reference number** and a **poll URL**; you check status by polling until it reaches a final state. You also configure a **result URL** and a **return URL**.
- **SDKs:** official libraries exist for Dart/Flutter, Python and Ruby; the Node.js package `pesepay-js` is unofficial (community-made), so for a NestJS backend consider calling the REST API through your own thin client, and review the library's code before trusting it with live keys.

Where Pesepay is used in Shofe:

1. Driver wallet top-up (to pay commission and keep receiving requests)
2. Passenger in-app payment for trips (later phase)
3. Driver payouts, if supported by your Pesepay account (confirm with them)

Safe implementation rules: keys stay on the server; amounts are computed server-side; every payment has an internal reference mapped to Pesepay's reference number; a background job polls pending payments; credit the wallet only after confirmed success; keep an immutable ledger; log failures for support.

Global note: Pesepay is oriented towards African markets. For other regions, enable Stripe or a local provider through the same abstraction.

6. Suggested Build Order (give Cursor one phase at a time)

Phase	Scope	Done when
0	Repo, Docker, DB schema, OpenAPI contract	`docker compose up` works; schema reviewed
1	Auth (OTP), users, roles, driver onboarding, admin approval	A driver can sign up and an admin can approve
2	Ride request, dispatch, offers, counter-offers, trip state machine	Full ride in a simulator, with cash
3	Live tracking, navigation, ratings, PIN verification	Real-time trip works on two phones
4	Driver wallet, commission ledger, Pesepay top-up	Top-up credits the wallet; commission deducts

Phase	Scope	Done when
5	Night Owl Mode (SOS, check-ins, Guardians, Quick-Help)	Night trip with SOS tested end to end
6	Admin dashboard, analytics, support tools	Staff can run daily operations
7	Localization, multi-country config, Stripe, hardening, pilot	Pilot with 20 to 50 drivers

7. Tips for Getting Good Results from Cursor

- Keep the master prompt and rules file in the repo so every session starts with the same context.
- Ask for **one phase or feature per request**, then run it and test before continuing.
- Ask Cursor to **write tests first** for money, state machines and dispatch logic.
- Review every payment and security change yourself; do not accept them blindly.
- Commit often so you can roll back a bad generation.
- Paste real error messages and ask Cursor to explain the cause before fixing.
- Verify third-party API details in the official docs. AI tools sometimes invent endpoints or fields.

API names and options reflect common industry choices at the time of writing. Check each provider's current pricing, coverage and terms before committing.